

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)*

Activity Tool : Case Study (Medium)

Assessment Tool Weightage: 30%

QUESTIONS		Total Marks: 50	
Q.No	Question	Bloom's Level	Marks
1	Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.	BL2 – Understand	5
2	A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.	BL3 – Apply	5

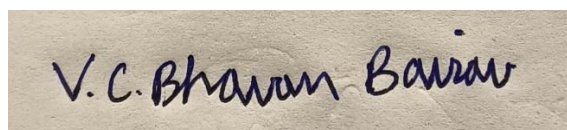
3	Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.	BL3 – Apply	5
4	Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.	BL3 – Apply	5
5	Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.	BL2 – Understand	5
6	Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.	BL3 – Apply	5
7	Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.	BL2 – Understand	5
8	A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.	BL3 – Apply	5
9	Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.	BL3 – Apply	5
10	Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.	BL3 – Apply	5

Code of Conduct:

I V.C.BHAVAN BAIRAV - 192511369 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



V.C. Bhavan Bairav

RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

1) Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.

AIM:

To identify the functional dependencies in the given Order relation and normalize it up to Third Normal Form (3NF).

GIVEN RELATION:

```
Database changed
mysql> CREATE TABLE ONLINE_ORDER (
  ->   OrderID INT,
  ->   CustID INT,
  ->   CustName VARCHAR(50),
  ->   ProdID INT,
  ->   ProdName VARCHAR(50),
  ->   Qty INT,
  ->   Price DECIMAL(10,2),
  ->   PRIMARY KEY (OrderID, ProdID)
  -> );
Query OK, 0 rows affected (0.05 sec)

mysql> INSERT INTO ONLINE_ORDER VALUES
  -> (1001, 501, 'Arun', 101, 'Laptop', 1, 55000.00),
  -> (1001, 501, 'Arun', 102, 'Mouse', 2, 800.00),
  -> (1002, 502, 'Meena', 103, 'Keyboard', 1, 1500.00),
  -> (1003, 503, 'Kiran', 101, 'Laptop', 1, 55000.00);
Query OK, 4 rows affected (0.02 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM ONLINE_ORDER;
+-----+-----+-----+-----+-----+-----+-----+
| OrderID | CustID | CustName | ProdID | ProdName | Qty | Price |
+-----+-----+-----+-----+-----+-----+-----+
| 1001 | 501 | Arun | 101 | Laptop | 1 | 55000.00 |
| 1001 | 501 | Arun | 102 | Mouse | 2 | 800.00 |
| 1002 | 502 | Meena | 103 | Keyboard | 1 | 1500.00 |
| 1003 | 503 | Kiran | 101 | Laptop | 1 | 55000.00 |
+-----+-----+-----+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

FUNCTIONAL DEPENDENCIES

- $\text{OrderID} \rightarrow \text{CustID}$
- $\text{CustID} \rightarrow \text{CustName}$
- $\text{ProdID} \rightarrow \text{ProdName, Price}$
- $(\text{OrderID}, \text{ProdID}) \rightarrow \text{Qty}$

CANDIDATE KEY

- $(\text{OrderID}, \text{ProdID})$

Since an order can contain multiple products, both OrderID and ProdID are required to uniquely identify the quantity.

IDENTIFY PARTIAL DEPENDENCIES

The candidate key is:

- $(\text{OrderID}, \text{ProdID})$

But:

- $\text{OrderID} \rightarrow \text{CustID}$
- $\text{CustID} \rightarrow \text{CustName}$
- $\text{ProdID} \rightarrow \text{ProdName, Price}$

CustID, CustName, ProdName, and Price do not depend on the complete composite key.

Therefore, partial dependencies exist.

3NF DECOMPOSITION

MySQL 8.0 Command Line Client

```
mysql> CREATE TABLE CUSTOMER (  
  ->   CustID INT PRIMARY KEY,  
  ->   CustName VARCHAR(50)  
  -> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> CREATE TABLE PRODUCT (  
  ->   ProdID INT PRIMARY KEY,  
  ->   ProdName VARCHAR(50),  
  ->   Price DECIMAL(10,2)  
  -> );  
Query OK, 0 rows affected (0.02 sec)  
  
mysql> CREATE TABLE ORDERS (  
  ->   OrderID INT PRIMARY KEY,  
  ->   CustID INT,  
  ->   FOREIGN KEY (CustID) REFERENCES CUSTOMER(CustID)  
  -> );  
Query OK, 0 rows affected (0.05 sec)  
  
mysql> CREATE TABLE ORDER_DETAILS (  
  ->   OrderID INT,  
  ->   ProdID INT,  
  ->   Qty INT,  
  ->   PRIMARY KEY (OrderID, ProdID),  
  ->   FOREIGN KEY (OrderID) REFERENCES ORDERS(OrderID),  
  ->   FOREIGN KEY (ProdID) REFERENCES PRODUCT(ProdID)  
  -> );  
Query OK, 0 rows affected (0.04 sec)  
  
mysql>
```

```
mysql> INSERT INTO CUSTOMER VALUES  
  -> (501, 'Arun'),  
  -> (502, 'Meena'),  
  -> (503, 'Kiran');  
Query OK, 3 rows affected (0.01 sec)  
Records: 3 Duplicates: 0 Warnings: 0  
  
mysql> INSERT INTO PRODUCT VALUES  
  -> (101, 'Laptop', 55000.00),  
  -> (102, 'Mouse', 800.00),  
  -> (103, 'Keyboard', 1500.00);  
Query OK, 3 rows affected (0.01 sec)  
Records: 3 Duplicates: 0 Warnings: 0  
  
mysql> INSERT INTO ORDERS VALUES  
  -> (1001, 501),  
  -> (1002, 502),  
  -> (1003, 503);  
Query OK, 3 rows affected (0.01 sec)  
Records: 3 Duplicates: 0 Warnings: 0  
  
mysql> INSERT INTO ORDER_DETAILS VALUES  
  -> (1001, 101, 1),  
  -> (1001, 102, 2),  
  -> (1002, 103, 1),  
  -> (1003, 101, 1);  
Query OK, 4 rows affected (0.01 sec)  
Records: 4 Duplicates: 0 Warnings: 0
```

MySQL 8.0 Command Line Client

Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM CUSTOMER;

CustID	CustName
501	Arun
502	Meena
503	Kiran

3 rows in set (0.00 sec)

mysql> SELECT * FROM PRODUCT;

ProdID	ProdName	Price
101	Laptop	55000.00
102	Mouse	800.00
103	Keyboard	1500.00

3 rows in set (0.00 sec)

mysql> SELECT * FROM ORDERS;

OrderID	CustID
1001	501
1002	502
1003	503

3 rows in set (0.00 sec)

mysql> SELECT * FROM ORDER_DETAILS;

OrderID	ProdID	Qty
1001	101	1
1001	102	2
1002	103	1
1003	101	1

4 rows in set (0.00 sec)

RESULT:

Thus, the functional dependencies in the online shopping Order relation were identified and the relation was successfully normalized into 3NF, reducing redundancy and preventing insertion, deletion, and update anomalies. The decomposition follows the same 3NF structure shown in the assessment material.

2) A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.

AIM:

To analyze the given hospital database relation, identify the functional dependencies and anomalies, and decompose the relation into Boyce-Codd Normal Form (BCNF) to reduce redundancy and improve data consistency.

GIVEN RELATION:

- Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName)

FUNCTIONAL DEPENDENCIES:

- $PID \rightarrow PName, DocID, WardNo$
- $DocID \rightarrow DocName, Specialization$
- $WardNo \rightarrow WardName$

ANOMALIES:

- **Update anomaly:** If a doctor's specialization changes, it may have to be changed in multiple patient records.
- **Insertion anomaly:** A new doctor cannot be stored unless a patient is assigned to that doctor.

- **Deletion anomaly:** Deleting the last patient of a doctor may remove information about that doctor.

BCNF DECOMPOSITION:

- Patient(PID, PName, DocID, WardNo)
- Doctor(DocID, DocName, Specialization)
- Ward(WardNo, WardName)

MySQL 8.0 Command Line Client

```
mysql> CREATE TABLE Patient (
->   PID INT PRIMARY KEY,
->   PName VARCHAR(30),
->   DocID INT,
->   WardNo INT
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Doctor (
->   DocID INT PRIMARY KEY,
->   DocName VARCHAR(30),
->   Specialization VARCHAR(40)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Ward (
->   WardNo INT PRIMARY KEY,
->   WardName VARCHAR(30)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO Patient VALUES
-> (101, 'Arun', 201, 1),
-> (102, 'Meena', 202, 2),
-> (103, 'Kiran', 201, 1),
-> (104, 'Ravi', 203, 3);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Doctor VALUES
-> (201, 'Dr.Raj', 'Cardiology'),
-> (202, 'Dr.Anita', 'Neurology'),
-> (203, 'Dr.Kumar', 'Orthopedics');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Ward VALUES
-> (1, 'General Ward'),
-> (2, 'ICU'),
-> (3, 'Emergency Ward');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM Patient;
+-----+-----+-----+-----+
| PID | PName | DocID | WardNo |
+-----+-----+-----+-----+
| 101 | Arun  | 201   | 1      |
| 102 | Meena | 202   | 2      |
| 103 | Kiran | 201   | 1      |
| 104 | Ravi  | 203   | 3      |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT * FROM Doctor;
+-----+-----+-----+
| DocID | DocName | Specialization |
+-----+-----+-----+
| 201   | Dr.Raj  | Cardiology     |
| 202   | Dr.Anita | Neurology      |
| 203   | Dr.Kumar | Orthopedics    |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Ward;
+-----+-----+
| WardNo | WardName |
+-----+-----+
| 1      | General Ward |
| 2      | ICU        |
| 3      | Emergency Ward |
+-----+-----+
3 rows in set (0.00 sec)
```

JUSTIFICATION

Each determinant becomes a key in its respective relation:

- PID is the key of Patient.
- DocID is the key of Doctor.
- WardNo is the key of Ward.

Therefore, the relations satisfy **BCNF**.

3) Case study: A university stores Enrollment (SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.

AIM:

To identify the functional dependencies in the Enrollment relation and decompose it into Second Normal Form (2NF) by removing partial dependencies.

GIVEN RELATION:

```
MySQL 8.0 Command Line Client

mysql> CREATE TABLE ENROLLMENT (
  ->   SID INT,
  ->   SName VARCHAR(30),
  ->   CourseID VARCHAR(10),
  ->   CourseName VARCHAR(40),
  ->   Faculty VARCHAR(30),
  ->   Grade VARCHAR(5),
  ->   PRIMARY KEY (SID, CourseID)
  -> );^Z
ERROR 1050 (42S01): Table 'enrollment' already exists
  -> INSERT INTO ENROLLMENT VALUES
  -> (101, 'Arun', 'C101', 'DBMS', 'Dr.Ravi', 'A'),
  -> (101, 'Arun', 'C102', 'DSA', 'Dr.Meena', 'B'),
  -> (102, 'Meena', 'C101', 'DBMS', 'Dr.Ravi', 'A'),
  -> (103, 'Kiran', 'C103', 'Networks', 'Dr.Suresh', 'B');
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '→
INSERT INTO ENROLLMENT VALUES
(101, 'Arun', 'C101', 'DBMS', 'Dr.Ravi', ' ' at line 1
mysql> INSERT INTO ENROLLMENT VALUES
  -> (101, 'Arun', 'C101', 'DBMS', 'Dr.Ravi', 'A'),
  -> (101, 'Arun', 'C102', 'DSA', 'Dr.Meena', 'B'),
  -> (102, 'Meena', 'C101', 'DBMS', 'Dr.Ravi', 'A'),
  -> (103, 'Kiran', 'C103', 'Networks', 'Dr.Suresh', 'B');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM ENROLLMENT;
+----+-----+-----+-----+-----+-----+
| SID | SName | CourseID | CourseName | Faculty | Grade |
+----+-----+-----+-----+-----+-----+
| 101 | Arun  | C101    | DBMS       | Dr.Ravi | A     |
| 101 | Arun  | C102    | DSA        | Dr.Meena | B     |
| 102 | Meena | C101    | DBMS       | Dr.Ravi | A     |
| 103 | Kiran | C103    | Networks   | Dr.Suresh | B     |
+----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

FUNCTIONAL DEPENDENCIES

- $SID \rightarrow SName$
- $CourseID \rightarrow CourseName, Faculty$
- $(SID, CourseID) \rightarrow Grade$

CANDIDATE KEY

- (SID, CourseID)

CREATE 2NF TABLES

```
mysql> CREATE TABLE STUDENT (  
->   SID INT PRIMARY KEY,  
->   SName VARCHAR(30)  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> CREATE TABLE COURSE (  
->   CourseID VARCHAR(10) PRIMARY KEY,  
->   CourseName VARCHAR(40),  
->   Faculty VARCHAR(30)  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> CREATE TABLE ENROLLMENT_2NF (  
->   SID INT,  
->   CourseID VARCHAR(10),  
->   Grade VARCHAR(5),  
->   PRIMARY KEY (SID, CourseID),  
->   FOREIGN KEY (SID) REFERENCES STUDENT(SID),  
->   FOREIGN KEY (CourseID) REFERENCES COURSE(CourseID)  
-> );  
Query OK, 0 rows affected (0.05 sec)  
  
mysql> INSERT INTO STUDENT VALUES  
-> (101, 'Arun'),  
-> (102, 'Meena'),  
-> (103, 'Kiran');  
Query OK, 3 rows affected (0.01 sec)  
Records: 3  Duplicates: 0  Warnings: 0  
  
mysql> INSERT INTO COURSE VALUES  
-> ('C101', 'DBMS', 'Dr.Ravi'),  
-> ('C102', 'DSA', 'Dr.Meena'),  
-> ('C103', 'Networks', 'Dr.Suresh');  
Query OK, 3 rows affected (0.01 sec)  
Records: 3  Duplicates: 0  Warnings: 0  
  
mysql> INSERT INTO ENROLLMENT_2NF VALUES  
-> (101, 'C101', 'A'),  
-> (101, 'C102', 'B'),  
-> (102, 'C101', 'A'),  
-> (103, 'C103', 'B');  
Query OK, 4 rows affected (0.01 sec)  
Records: 4  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM STUDENT;
+-----+-----+
| SID | SName |
+-----+-----+
| 101 | Arun  |
| 102 | Meena |
| 103 | Kiran |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM COURSE;
+-----+-----+-----+
| CourseID | CourseName | Faculty |
+-----+-----+-----+
| C101     | DBMS       | Dr.Ravi |
| C102     | DSA        | Dr.Meena |
| C103     | Networks   | Dr.Suresh |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM ENROLLMENT_2NF;
+-----+-----+-----+
| SID | CourseID | Grade |
+-----+-----+-----+
| 101 | C101     | A     |
| 101 | C102     | B     |
| 102 | C101     | A     |
| 103 | C103     | B     |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

RESULT:

Thus, the Enrollment relation was successfully decomposed into 2NF by removing the partial dependencies. This matches the decomposition specified for Q3 in your assessment.

4) Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.

AIM:

To identify the functional dependencies in the airline Booking relation and normalize it up to Boyce-Codd Normal Form (BCNF).

GIVEN RELATION:

```
mysql> CREATE TABLE AIRLINE_BOOKING (
->     FlightNo VARCHAR(10),
->     PassID VARCHAR(10),
->     PassName VARCHAR(30),
->     Seat VARCHAR(10),
->     DeptCity VARCHAR(30),
->     ArrCity VARCHAR(30),
->     BookingDate DATE,
->     PRIMARY KEY (FlightNo, PassID)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO AIRLINE_BOOKING VALUES
-> ('F101','P101','Arun','12A','Chennai','Delhi','2026-08-10'),
-> ('F101','P102','Meena','12B','Chennai','Delhi','2026-08-10'),
-> ('F102','P103','Kiran','15A','Mumbai','Bangalore','2026-08-11'),
-> ('F103','P101','Arun','20C','Delhi','Kolkata','2026-08-12');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM AIRLINE_BOOKING;
+-----+-----+-----+-----+-----+-----+-----+
| FlightNo | PassID | PassName | Seat | DeptCity | ArrCity | BookingDate |
+-----+-----+-----+-----+-----+-----+-----+
| F101     | P101   | Arun     | 12A  | Chennai  | Delhi   | 2026-08-10   |
| F101     | P102   | Meena    | 12B  | Chennai  | Delhi   | 2026-08-10   |
| F102     | P103   | Kiran    | 15A  | Mumbai   | Bangalore | 2026-08-11   |
| F103     | P101   | Arun     | 20C  | Delhi    | Kolkata  | 2026-08-12   |
+-----+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

FUNCTIONAL DEPENDENCIES

- $\text{PassID} \rightarrow \text{PassName}$
- $\text{FlightNo} \rightarrow \text{DeptCity}, \text{ArrCity}, \text{Date}$
- $(\text{FlightNo}, \text{PassID}) \rightarrow \text{Seat}$

CANDIDATE KEY

- (FlightNo, PassID)

Passenger information depends only on PassID, while flight information depends only on FlightNo. Hence, they are partial dependencies with respect to the composite key.

BCNF DECOMPOSITION

MySQL 8.0 Command Line Client

```
mysql> CREATE TABLE PASSENGER (  
->   PassID VARCHAR(10) PRIMARY KEY,  
->   PassName VARCHAR(30)  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> CREATE TABLE FLIGHT (  
->   FlightNo VARCHAR(10) PRIMARY KEY,  
->   DeptCity VARCHAR(30),  
->   ArrCity VARCHAR(30),  
->   FlightDate DATE  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> CREATE TABLE BOOKING_BCNF (  
->   FlightNo VARCHAR(10),  
->   PassID VARCHAR(10),  
->   Seat VARCHAR(10),  
->   PRIMARY KEY (FlightNo, PassID),  
->   FOREIGN KEY (FlightNo) REFERENCES FLIGHT(FlightNo),  
->   FOREIGN KEY (PassID) REFERENCES PASSENGER(PassID)  
-> );  
Query OK, 0 rows affected (0.06 sec)  
  
mysql> INSERT INTO PASSENGER VALUES  
-> ('P101','Arun'),  
-> ('P102','Meena'),  
-> ('P103','Kiran');  
Query OK, 3 rows affected (0.01 sec)  
Records: 3 Duplicates: 0 Warnings: 0  
  
mysql> INSERT INTO FLIGHT VALUES  
-> ('F101','Chennai','Delhi','2026-08-10'),  
-> ('F102','Mumbai','Bangalore','2026-08-11'),  
-> ('F103','Delhi','Kolkata','2026-08-12');  
Query OK, 3 rows affected (0.01 sec)  
Records: 3 Duplicates: 0 Warnings: 0  
  
mysql> INSERT INTO BOOKING_BCNF VALUES  
-> ('F101','P101','12A'),  
-> ('F101','P102','12B'),  
-> ('F102','P103','15A'),  
-> ('F103','P101','20C');  
Query OK, 4 rows affected (0.01 sec)  
Records: 4 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM PASSENGER;
+-----+-----+
| PassID | PassName |
+-----+-----+
| P101   | Arun     |
| P102   | Meena    |
| P103   | Kiran    |
+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM FLIGHT;
+-----+-----+-----+-----+
| FlightNo | DeptCity | ArrCity | FlightDate |
+-----+-----+-----+-----+
| F101     | Chennai | Delhi   | 2026-08-10 |
| F102     | Mumbai  | Bangalore | 2026-08-11 |
| F103     | Delhi   | Kolkata | 2026-08-12 |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM BOOKING_BCNF;
+-----+-----+-----+
| FlightNo | PassID | Seat |
+-----+-----+-----+
| F101     | P101   | 12A  |
| F101     | P102   | 12B  |
| F102     | P103   | 15A  |
| F103     | P101   | 20C  |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

RESULT:

Thus, the airline booking relation was successfully normalized up to BCNF by removing the partial dependencies and separating passenger, flight, and booking information.

5) Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.

AIM:

To identify the functional and transitive dependencies in an HR system relation and normalize it up to Third Normal Form (3NF).

GIVEN RELATION:

MySQL 8.0 Command Line Client

```
mysql> CREATE TABLE HR_EMPLOYEE (
->   EmpID VARCHAR(10) PRIMARY KEY,
->   EmpName VARCHAR(50),
->   DeptID VARCHAR(10),
->   DeptName VARCHAR(50),
->   ManagerID VARCHAR(10),
->   ManagerName VARCHAR(50),
->   CountryID VARCHAR(10),
->   CountryName VARCHAR(50),
->   JobID VARCHAR(10),
->   JobTitle VARCHAR(50),
->   Salary DECIMAL(10,2)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO HR_EMPLOYEE VALUES
-> ('E101','Arun','D01','Engineering','M01','Ravi','C01','India','J01','Software Engineer',55000),
-> ('E102','Meena','D01','Engineering','M01','Ravi','C01','India','J02','Senior Engineer',70000),
-> ('E103','John','D02','Finance','M02','David','C02','USA','J03','Financial Analyst',65000),
-> ('E104','Sarah','D03','Human Resources','M03','Emma','C03','UK','J04','HR Executive',60000);
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM HR_EMPLOYEE;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| EmpID | EmpName | DeptID | DeptName | ManagerID | ManagerName | CountryID | CountryName | JobID | JobTitle | Salary |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| E101 | Arun | D01 | Engineering | M01 | Ravi | C01 | India | J01 | Software Engineer | 55000.00 |
| E102 | Meena | D01 | Engineering | M01 | Ravi | C01 | India | J02 | Senior Engineer | 70000.00 |
| E103 | John | D02 | Finance | M02 | David | C02 | USA | J03 | Financial Analyst | 65000.00 |
| E104 | Sarah | D03 | Human Resources | M03 | Emma | C03 | UK | J04 | HR Executive | 60000.00 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> _
```

FUNCTIONAL DEPENDENCIES:

- $\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}, \text{ManagerID}, \text{CountryID}, \text{JobID}, \text{Salary}$
- $\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$
- $\text{ManagerID} \rightarrow \text{ManagerName}$
- $\text{CountryID} \rightarrow \text{CountryName}$
- $\text{JobID} \rightarrow \text{JobTitle}$

CANDIDATE KEY

- EmpID

TRANSITIVE DEPENDENCIES

The following are transitive dependencies:

- $\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{DeptName}$
- $\text{EmpID} \rightarrow \text{ManagerID} \rightarrow \text{ManagerName}$
- $\text{EmpID} \rightarrow \text{CountryID} \rightarrow \text{CountryName}$
- $\text{EmpID} \rightarrow \text{JobID} \rightarrow \text{JobTitle}$

Therefore, attributes such as DeptName, ManagerName, CountryName, and JobTitle depend indirectly on EmpID.

3NF DECOMPOSITION

 MySQL 8.0 Command Line Client

```
mysql> use aug;
Database changed
mysql> CREATE TABLE EMPLOYEE (
  ->     EmpID VARCHAR(10) PRIMARY KEY,
  ->     EmpName VARCHAR(50),
  ->     DeptID VARCHAR(10),
  ->     ManagerID VARCHAR(10),
  ->     CountryID VARCHAR(10),
  ->     JobID VARCHAR(10),
  ->     Salary DECIMAL(10,2)
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE DEPARTMENT (
  ->     DeptID VARCHAR(10) PRIMARY KEY,
  ->     DeptName VARCHAR(50),
  ->     ManagerID VARCHAR(10)
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE MANAGER (
  ->     ManagerID VARCHAR(10) PRIMARY KEY,
  ->     ManagerName VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE COUNTRY (
  ->     CountryID VARCHAR(10) PRIMARY KEY,
  ->     CountryName VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE JOB (
  ->     JobID VARCHAR(10) PRIMARY KEY,
  ->     JobTitle VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> INSERT INTO MANAGER VALUES
  -> ('M01','Ravi'),
  -> ('M02','David'),
  -> ('M03','Emma');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO COUNTRY VALUES
  -> ('C01','India'),
  -> ('C02','USA'),
  -> ('C03','UK');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO JOB VALUES
  -> ('J01','Software Engineer'),
  -> ('J02','Senior Engineer'),
  -> ('J03','Financial Analyst'),
  -> ('J04','HR Executive');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO DEPARTMENT VALUES
  -> ('D01','Engineering','M01'),
  -> ('D02','Finance','M02'),
  -> ('D03','Human Resources','M03');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO EMPLOYEE VALUES
  -> ('E101','Arun','D01','M01','C01','J01',55000),
  -> ('E102','Meena','D01','M01','C01','J02',70000),
  -> ('E103','John','D02','M02','C02','J03',65000),
  -> ('E104','Sarah','D03','M03','C03','J04',60000);
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

MySQL 8.0 Command Line Client

```
mysql> SELECT * FROM EMPLOYEE;
```

EmpID	EmpName	DeptID	ManagerID	CountryID	JobID	Salary
E101	Arun	D01	M01	C01	J01	55000.00
E102	Meena	D01	M01	C01	J02	70000.00
E103	John	D02	M02	C02	J03	65000.00
E104	Sarah	D03	M03	C03	J04	60000.00

```
4 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM DEPARTMENT;
```

DeptID	DeptName	ManagerID
D01	Engineering	M01
D02	Finance	M02
D03	Human Resources	M03

```
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM MANAGER;
```

ManagerID	ManagerName
M01	Ravi
M02	David
M03	Emma

```
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM COUNTRY;
```

CountryID	CountryName
C01	India
C02	USA
C03	UK

```
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM JOB;
```

JobID	JobTitle
J01	Software Engineer
J02	Senior Engineer
J03	Financial Analyst
J04	HR Executive

RESULT:

Thus, the multinational HR system relation was successfully normalized up to 3NF by identifying all functional and transitive dependencies and decomposing the relation into EMPLOYEE, DEPARTMENT, MANAGER, COUNTRY, and JOB tables. This reduces data redundancy and prevents insertion, update, and deletion anomalies.

6) Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.

AIM:

To identify the multivalued dependencies in the Library relation and normalize it up to Fourth Normal Form (4NF).

GIVEN RELATION:

Select MySQL 8.0 Command Line Client

```
mysql> CREATE TABLE LIBRARY (
->   MemberID VARCHAR(10),
->   MemberName VARCHAR(50),
->   BookID VARCHAR(10),
->   Title VARCHAR(100),
->   Author VARCHAR(50),
->   Genre VARCHAR(30),
->   BorrowDate DATE,
->   PRIMARY KEY (MemberID, BookID, BorrowDate)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO LIBRARY VALUES
-> ('M101','Arun','B101','Database Systems','Korth','DBMS','2026-08-01'),
-> ('M101','Arun','B102','Operating Systems','Galvin','OS','2026-08-03'),
-> ('M101','Arun','B103','Computer Networks','Tanenbaum','Networks','2026-08-05'),
-> ('M102','Meena','B101','Database Systems','Korth','DBMS','2026-08-02');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM LIBRARY;
+-----+-----+-----+-----+-----+-----+-----+
| MemberID | MemberName | BookID | Title           | Author   | Genre  | BorrowDate |
+-----+-----+-----+-----+-----+-----+-----+
| M101     | Arun       | B101   | Database Systems | Korth    | DBMS   | 2026-08-01 |
| M101     | Arun       | B102   | Operating Systems | Galvin   | OS     | 2026-08-03 |
| M101     | Arun       | B103   | Computer Networks | Tanenbaum | Networks | 2026-08-05 |
| M102     | Meena      | B101   | Database Systems | Korth    | DBMS   | 2026-08-02 |
+-----+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

MULTIVALUED DEPENDENCIES

Assume that a member can borrow **multiple books**, and a book can have multiple independent genres/authors associated with it.

The important multivalued dependencies are:

- $\text{MemberID} \twoheadrightarrow \text{BookID}$
- $\text{MemberID} \twoheadrightarrow \text{BorrowDate}$

This means a member may have multiple books and multiple borrowing dates independently.

CREATE 4NF TABLES

```
MySQL 8.0 Command Line Client

mysql> CREATE TABLE MEMBER (
  ->   MemberID VARCHAR(10) PRIMARY KEY,
  ->   MemberName VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE BOOK (
  ->   BookID VARCHAR(10) PRIMARY KEY,
  ->   Title VARCHAR(100),
  ->   Author VARCHAR(50),
  ->   Genre VARCHAR(30)
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE BORROW (
  ->   MemberID VARCHAR(10),
  ->   BookID VARCHAR(10),
  ->   BorrowDate DATE,
  ->   PRIMARY KEY (MemberID, BookID, BorrowDate),
  ->   FOREIGN KEY (MemberID) REFERENCES MEMBER(MemberID),
  ->   FOREIGN KEY (BookID) REFERENCES BOOK(BookID)
  -> );
Query OK, 0 rows affected (0.05 sec)

mysql> INSERT INTO MEMBER VALUES
  -> ('M101','Arun'),
  -> ('M102','Meena');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> INSERT INTO BOOK VALUES
  -> ('B101','Database Systems','Korth','DBMS'),
  -> ('B102','Operating Systems','Galvin','OS'),
  -> ('B103','Computer Networks','Tanenbaum','Networks');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO BORROW VALUES
  -> ('M101','B101','2026-08-01'),
  -> ('M101','B102','2026-08-03'),
  -> ('M101','B103','2026-08-05'),
  -> ('M102','B101','2026-08-02');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM MEMBER;
+-----+-----+
| MemberID | MemberName |
+-----+-----+
| M101     | Arun       |
| M102     | Meena      |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM BOOK;
+-----+-----+-----+-----+
| BookID | Title           | Author   | Genre  |
+-----+-----+-----+-----+
| B101   | Database Systems | Korth    | DBMS   |
| B102   | Operating Systems | Galvin   | OS     |
| B103   | Computer Networks | Tanenbaum | Networks |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM BORROW;
+-----+-----+-----+
| MemberID | BookID | BorrowDate |
+-----+-----+-----+
| M101     | B101   | 2026-08-01 |
| M102     | B101   | 2026-08-02 |
| M101     | B102   | 2026-08-03 |
| M101     | B103   | 2026-08-05 |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

RESULT:

Thus, the Library relation was successfully decomposed into 4NF by identifying the multivalued dependencies and separating member, book, and borrowing information. This eliminates redundancy caused by independent multivalued facts and prevents modification anomalies.

7) Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.

AIM:

To analyze a telecom billing relation and identify all candidate keys, primary key, and functional dependencies before normalization.

GIVEN RELATION:

```
MySQL 8.0 Command Line Client
mysql> CREATE TABLE TELECOM_BILLING (
->   CustomerID VARCHAR(10),
->   CustomerName VARCHAR(50),
->   PhoneNo VARCHAR(15),
->   PlanID VARCHAR(10),
->   PlanName VARCHAR(50),
->   PlanType VARCHAR(30),
->   MonthlyCharge DECIMAL(10,2),
->   BillID VARCHAR(10),
->   BillDate DATE,
->   DueDate DATE,
->   UsageID VARCHAR(10),
->   CallMinutes INT,
->   DataUsed DECIMAL(10,2),
->   SMSCount INT,
->   BillAmount DECIMAL(10,2),
->
->   PRIMARY KEY (BillID, UsageID)
-> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO TELECOM_BILLING VALUES
-> ('C101','Arun','9876543210','P101','Smart Plan','Postpaid',
-> 499.00,'B101','2026-08-01','2026-08-15','U101',
-> 250,12.5,100,699.00);
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> INSERT INTO TELECOM_BILLING VALUES
-> ('C102','Meena','9876543211','P102','Premium Plan','Postpaid',
-> 799.00,'B102','2026-08-01','2026-08-15','U102',
-> 400,25.0,200,999.00);
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> INSERT INTO TELECOM_BILLING VALUES
-> ('C103','Kiran','9876543212','P103','Basic Plan','Prepaid',
-> 299.00,'B103','2026-08-01','2026-08-15','U103',
-> 150,8.5,75,399.00);
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> INSERT INTO TELECOM_BILLING VALUES
-> ('C101','Arun','9876543210','P101','Smart Plan','Postpaid',
-> 499.00,'B104','2026-09-01','2026-09-15','U104',
-> 300,15.0,120,749.00);
Query OK, 1 row affected (0.00 sec)
```

```
mysql> SELECT * FROM TELECOM_BILLING;
```

CustomerID	CustomerName	Phonello	PlanID	PlanName	PlanType	MonthlyCharge	BillID	BillDate	DueDate	UsageID	CallMinutes	DataUsed	SMSCount	BillAmount
C101	Arun	9876543210	P101	Smart Plan	Postpaid	499.00	B101	2026-08-01	2026-08-15	U101	250	12.50	100	699.00
C102	Meena	9876543211	P102	Premium Plan	Postpaid	799.00	B102	2026-08-01	2026-08-15	U102	400	25.00	200	999.00
C103	Kiran	9876543212	P103	Basic Plan	Prepaid	299.00	B103	2026-08-01	2026-08-15	U103	150	8.50	75	399.00
C101	Arun	9876543210	P101	Smart Plan	Postpaid	499.00	B104	2026-09-01	2026-09-15	U104	300	15.00	120	749.00

```
4 rows in set (0.00 sec)
```

FUNCTIONAL DEPENDENCIES

- $\text{CustomerID} \rightarrow \text{CustomerName}, \text{PhoneNo}, \text{PlanID}$
- $\text{PhoneNo} \rightarrow \text{CustomerID}, \text{CustomerName}, \text{PlanID}$
- $\text{PlanID} \rightarrow \text{PlanName}, \text{PlanType}, \text{MonthlyCharge}$
- $\text{BillID} \rightarrow \text{CustomerID}, \text{BillDate}, \text{DueDate}, \text{BillAmount}$
- $\text{UsageID} \rightarrow \text{CustomerID}, \text{PhoneNo}, \text{CallMinutes}, \text{DataUsed}, \text{SMSCount}$
- $(\text{CustomerID}, \text{BillDate}) \rightarrow \text{BillID}, \text{DueDate}, \text{BillAmount}$

EXPLANATION OF FUNCTIONAL DEPENDENCIES:

- $\text{CustomerID} \rightarrow \text{CustomerName}, \text{PhoneNo}, \text{PlanID}$

Each customer has a unique customer ID, which determines the customer's name, phone number, and subscribed plan.

- $\text{PhoneNo} \rightarrow \text{CustomerID}, \text{CustomerName}, \text{PlanID}$

Each registered phone number belongs to one customer.

- $\text{PlanID} \rightarrow \text{PlanName}, \text{PlanType}, \text{MonthlyCharge}$

Each plan ID uniquely identifies the plan details and monthly charge.

- $\text{BillID} \rightarrow \text{CustomerID}, \text{BillDate}, \text{DueDate}, \text{BillAmount}$

Each bill ID uniquely identifies a customer's bill and its billing details.

- $\text{UsageID} \rightarrow \text{CustomerID}, \text{PhoneNo}, \text{CallMinutes}, \text{DataUsed}, \text{SMSCount}$

Each usage record contains the usage information for a particular customer.

- $(\text{CustomerID}, \text{BillDate}) \rightarrow \text{BillID}, \text{DueDate}, \text{BillAmount}$

For a customer, a particular billing date identifies one bill.

CANDIDATE KEYS

The candidate keys are:

- CustomerID
- PhoneNo
- BillID
- UsageID

For the complete billing record, the key can be represented by:

- $(\text{BillID}, \text{UsageID})$

depending on the assumption that a bill can contain multiple usage records.

PRIMARY KEY

A suitable primary key for the original relation is:

- BillID

However, if the table stores individual usage records associated with a bill, then:

- $(\text{BillID}, \text{UsageID})$

can be used as the composite primary key.

RESULT:

Thus, the telecom billing schema was analyzed successfully. The candidate keys, primary key, and functional dependencies were identified before normalization. These dependencies can be used in the next step to identify partial and transitive dependencies and normalize the schema up to 3NF/BCNF.

8) A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.

AIM:

To identify the functional dependencies in the retail inventory relation and apply complete normalization up to Third Normal Form (3NF).

GIVEN RELATION:

```
MySQL 8.0 Command Line Client

mysql> CREATE TABLE PRODUCT_INVENTORY (
  ->   PID VARCHAR(10) PRIMARY KEY,
  ->   PName VARCHAR(50),
  ->   SupplierID VARCHAR(10),
  ->   SupplierName VARCHAR(50),
  ->   Category VARCHAR(30),
  ->   Price DECIMAL(10,2),
  ->   Warehouse VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO PRODUCT_INVENTORY VALUES
  -> ('P101','Laptop','S101','Tech Supplies Ltd','Electronics',55000.00,'Chennai Warehouse');
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> INSERT INTO PRODUCT_INVENTORY VALUES
  -> ('P102','Keyboard','S102','Office Solutions','Accessories',1200.00,'Chennai Warehouse');
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> INSERT INTO PRODUCT_INVENTORY VALUES
  -> ('P103','Mouse','S101','Tech Supplies Ltd','Accessories',800.00,'Bangalore Warehouse');
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> INSERT INTO PRODUCT_INVENTORY VALUES
  -> ('P104','Monitor','S103','Display World','Electronics',15000.00,'Mumbai Warehouse');
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM PRODUCT_INVENTORY;
+-----+-----+-----+-----+-----+-----+-----+
| PID | PName | SupplierID | SupplierName | Category | Price | Warehouse |
+-----+-----+-----+-----+-----+-----+-----+
| P101 | Laptop | S101 | Tech Supplies Ltd | Electronics | 55000.00 | Chennai Warehouse |
| P102 | Keyboard | S102 | Office Solutions | Accessories | 1200.00 | Chennai Warehouse |
| P103 | Mouse | S101 | Tech Supplies Ltd | Accessories | 800.00 | Bangalore Warehouse |
| P104 | Monitor | S103 | Display World | Electronics | 15000.00 | Mumbai Warehouse |
+-----+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

NORMALIZATION

Select MySQL 8.0 Command Line Client

```
mysql> CREATE TABLE SUPPLIER (  
-> SupplierID VARCHAR(10) PRIMARY KEY,  
-> SupplierName VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> CREATE TABLE PRODUCT (  
-> PID VARCHAR(10) PRIMARY KEY,  
-> PName VARCHAR(50),  
-> SupplierID VARCHAR(10),  
-> Category VARCHAR(30),  
-> Price DECIMAL(10,2),  
-> Warehouse VARCHAR(50),  
-> FOREIGN KEY (SupplierID) REFERENCES SUPPLIER(SupplierID)  
-> );  
Query OK, 0 rows affected (0.06 sec)  
  
mysql> INSERT INTO SUPPLIER VALUES  
-> ('S101','Tech Supplies Ltd'),  
-> ('S102','Office Solutions'),  
-> ('S103','Display World');  
Query OK, 3 rows affected (0.01 sec)  
Records: 3 Duplicates: 0 Warnings: 0  
  
mysql> INSERT INTO PRODUCT VALUES  
-> ('P101','Laptop','S101','Electronics',55000.00,'Chennai Warehouse'),  
-> ('P102','Keyboard','S102','Accessories',1200.00,'Chennai Warehouse'),  
-> ('P103','Mouse','S101','Accessories',800.00,'Bangalore Warehouse'),  
-> ('P104','Monitor','S103','Electronics',15000.00,'Mumbai Warehouse');  
Query OK, 4 rows affected (0.01 sec)  
Records: 4 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM SUPPLIER;  
+-----+-----+  
| SupplierID | SupplierName |  
+-----+-----+  
| S101      | Tech Supplies Ltd |  
| S102      | Office Solutions |  
| S103      | Display World |  
+-----+-----+  
3 rows in set (0.00 sec)  
  
mysql> SELECT * FROM PRODUCT;  
+-----+-----+-----+-----+-----+-----+  
| PID | PName | SupplierID | Category | Price | Warehouse |  
+-----+-----+-----+-----+-----+-----+  
| P101 | Laptop | S101 | Electronics | 55000.00 | Chennai Warehouse |  
| P102 | Keyboard | S102 | Accessories | 1200.00 | Chennai Warehouse |  
| P103 | Mouse | S101 | Accessories | 800.00 | Bangalore Warehouse |  
| P104 | Monitor | S103 | Electronics | 15000.00 | Mumbai Warehouse |  
+-----+-----+-----+-----+-----+-----+  
4 rows in set (0.00 sec)
```

RESULT:

Thus, the retail inventory relation was successfully normalized up to 3NF by identifying the functional and transitive dependencies and separating supplier information into a separate SUPPLIER table. This reduces data redundancy and prevents update, insertion, and deletion anomalies.

9) Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.

AIM:

To analyze the poorly normalized School ERP schema and decompose it into relations up to BCNF, eliminating partial and transitive dependencies and reducing redundancy.

GIVEN RELATION:

```
MySQL 8.0 Command Line Client

mysql> CREATE TABLE SCHOOL_ERP (
  ->   StudentID VARCHAR(10),
  ->   StudentName VARCHAR(50),
  ->   ClassID VARCHAR(10),
  ->   ClassName VARCHAR(30),
  ->   TeacherID VARCHAR(10),
  ->   TeacherName VARCHAR(50),
  ->   SubjectID VARCHAR(10),
  ->   SubjectName VARCHAR(50),
  ->   ExamID VARCHAR(10),
  ->   ExamName VARCHAR(50),
  ->   Marks INT,
  ->
  ->   PRIMARY KEY (StudentID, ExamID)
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO SCHOOL_ERP VALUES
  -> ('ST101','Arun','C01','CSE-A','T01','Ravi',
  -> 'SUB01','DBMS','EX01','Internal Test 1',85);
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> INSERT INTO SCHOOL_ERP VALUES
  -> ('ST101','Arun','C01','CSE-A','T02','Meena',
  -> 'SUB02','Operating Systems','EX02','Internal Test 1',78);
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> INSERT INTO SCHOOL_ERP VALUES
  -> ('ST102','Kiran','C01','CSE-A','T01','Ravi',
  -> 'SUB01','DBMS','EX01','Internal Test 1',90);
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> INSERT INTO SCHOOL_ERP VALUES
  -> ('ST103','Meena','C02','CSE-B','T03','David',
  -> 'SUB03','Computer Networks','EX03','Internal Test 2',88);
Query OK, 1 row affected (0.01 sec)
```

```
mysql> SELECT * FROM SCHOOL_ERP;
```

StudentID	StudentName	ClassID	ClassName	TeacherID	TeacherName	SubjectID	SubjectName	ExamID	ExamName	Marks
ST101	Arun	C01	CSE-A	T01	Ravi	SUB01	DBMS	EX01	Internal Test 1	85
ST101	Arun	C01	CSE-A	T02	Meena	SUB02	Operating Systems	EX02	Internal Test 1	78
ST102	Kiran	C01	CSE-A	T01	Ravi	SUB01	DBMS	EX01	Internal Test 1	90
ST103	Meena	C02	CSE-B	T03	David	SUB03	Computer Networks	EX03	Internal Test 2	88

4 rows in set (0.00 sec)

FUNCTIONAL DEPENDENCIES

- $\text{StudentID} \rightarrow \text{StudentName}$
- $\text{ClassID} \rightarrow \text{ClassName}, \text{TeacherID}$
- $\text{TeacherID} \rightarrow \text{TeacherName}$
- $\text{SubjectID} \rightarrow \text{SubjectName}$
- $\text{ExamID} \rightarrow \text{ExamName}, \text{ClassID}, \text{SubjectID}$
- $(\text{StudentID}, \text{ExamID}) \rightarrow \text{Marks}$

CANDIDATE KEY

- $(\text{StudentID}, \text{ExamID})$

A student can have multiple examinations, while a particular student and examination combination uniquely identifies the marks obtained.

BCNF DECOMPOSITION

CREATION:

MySQL 8.0 Command Line Client

```
mysql> CREATE TABLE STUDENT (  
  ->     StudentID VARCHAR(10) PRIMARY KEY,  
  ->     StudentName VARCHAR(50)  
  -> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> CREATE TABLE TEACHER (  
  ->     TeacherID VARCHAR(10) PRIMARY KEY,  
  ->     TeacherName VARCHAR(50)  
  -> );  
Query OK, 0 rows affected (0.02 sec)  
  
mysql> CREATE TABLE CLASS (  
  ->     ClassID VARCHAR(10) PRIMARY KEY,  
  ->     ClassName VARCHAR(30),  
  ->     TeacherID VARCHAR(10),  
  ->     FOREIGN KEY (TeacherID) REFERENCES TEACHER(TeacherID)  
  -> );  
Query OK, 0 rows affected (0.04 sec)  
  
mysql> CREATE TABLE SUBJECT (  
  ->     SubjectID VARCHAR(10) PRIMARY KEY,  
  ->     SubjectName VARCHAR(50)  
  -> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> CREATE TABLE EXAM (  
  ->     ExamID VARCHAR(10) PRIMARY KEY,  
  ->     ExamName VARCHAR(50),  
  ->     ClassID VARCHAR(10),  
  ->     SubjectID VARCHAR(10),  
  ->     FOREIGN KEY (ClassID) REFERENCES CLASS(ClassID),  
  ->     FOREIGN KEY (SubjectID) REFERENCES SUBJECT(SubjectID)  
  -> );  
Query OK, 0 rows affected (0.05 sec)  
  
mysql> CREATE TABLE STUDENT_EXAM (  
  ->     StudentID VARCHAR(10),  
  ->     ExamID VARCHAR(10),  
  ->     Marks INT,  
  ->     PRIMARY KEY (StudentID, ExamID),  
  ->     FOREIGN KEY (StudentID) REFERENCES STUDENT(StudentID),  
  ->     FOREIGN KEY (ExamID) REFERENCES EXAM(ExamID)  
  -> );  
Query OK, 0 rows affected (0.04 sec)
```

INSERTION:

```
mysql> INSERT INTO STUDENT VALUES
  -> ('ST101','Arun'),
  -> ('ST102','Kiran'),
  -> ('ST103','Meena');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO TEACHER VALUES
  -> ('T01','Ravi'),
  -> ('T02','Meena'),
  -> ('T03','David');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO CLASS VALUES
  -> ('C01','CSE-A','T01'),
  -> ('C02','CSE-B','T03');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> INSERT INTO SUBJECT VALUES
  -> ('SUB01','DBMS'),
  -> ('SUB02','Operating Systems'),
  -> ('SUB03','Computer Networks');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO EXAM VALUES
  -> ('EX01','Internal Test 1','C01','SUB01'),
  -> ('EX02','Internal Test 1','C01','SUB02'),
  -> ('EX03','Internal Test 2','C02','SUB03');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO STUDENT_EXAM VALUES
  -> ('ST101','EX01',85),
  -> ('ST101','EX02',78),
  -> ('ST102','EX01',90),
  -> ('ST103','EX03',88);
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

SELECTION:

MySQL 8.0 Command Line Client

```
mysql> SELECT * FROM STUDENT;
```

StudentID	StudentName
ST101	Arun
ST102	Kiran
ST103	Meena

```
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM TEACHER;
```

TeacherID	TeacherName
T01	Ravi
T02	Meena
T03	David

```
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM CLASS;
```

ClassID	ClassName	TeacherID
C01	CSE-A	T01
C02	CSE-B	T03

```
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM SUBJECT;
```

SubjectID	SubjectName
SUB01	DBMS
SUB02	Operating Systems
SUB03	Computer Networks

```
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM EXAM;
```

ExamID	ExamName	ClassID	SubjectID
EX01	Internal Test 1	C01	SUB01
EX02	Internal Test 1	C01	SUB02
EX03	Internal Test 2	C02	SUB03

```
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM STUDENT_EXAM;
```

StudentID	ExamID	Marks
ST101	EX01	85
ST101	EX02	78
ST102	EX01	90
ST103	EX03	88

```
4 rows in set (0.00 sec)
```

RESULT:

Thus, the poorly normalized School ERP schema was successfully analyzed and decomposed into BCNF relations. The decomposition removes partial and transitive dependencies, ensures that every determinant is a candidate key, and reduces redundancy while maintaining the relationships between students, classes, teachers, subjects, examinations, and marks.

10) Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.

AIM:

To identify the join dependencies (JDs) in a movie streaming platform relation and decompose it into 5NF while verifying that the decomposition is lossless.

GIVEN RELATION:

```
MySQL 8.0 Command Line Client

mysql> CREATE TABLE MOVIE_STREAMING (
  ->   MovieID VARCHAR(10),
  ->   ActorID VARCHAR(10),
  ->   PlatformID VARCHAR(10),
  ->   PRIMARY KEY (MovieID, ActorID, PlatformID)
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO MOVIE_STREAMING VALUES
  -> ('M101','A101','P01'),
  -> ('M101','A102','P01'),
  -> ('M102','A101','P01'),
  -> ('M102','A103','P02'),
  -> ('M103','A102','P01');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM MOVIE_STREAMING;
+-----+-----+-----+
| MovieID | ActorID | PlatformID |
+-----+-----+-----+
| M101    | A101    | P01        |
| M101    | A102    | P01        |
| M102    | A101    | P01        |
| M102    | A103    | P02        |
| M103    | A102    | P01        |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

JOIN DEPENDENCIES

- *(MovieID, ActorID),
- (MovieID, PlatformID),
- (ActorID, PlatformID)

5NF DECOMPOSITION

MySQL 8.0 Command Line Client

```
mysql> CREATE TABLE MOVIE_ACTOR (  
->   MovieID VARCHAR(10),  
->   ActorID VARCHAR(10),  
->   PRIMARY KEY (MovieID, ActorID)  
-> );  
Query OK, 0 rows affected (0.02 sec)  
  
mysql> CREATE TABLE MOVIE_PLATFORM (  
->   MovieID VARCHAR(10),  
->   PlatformID VARCHAR(10),  
->   PRIMARY KEY (MovieID, PlatformID)  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> CREATE TABLE ACTOR_PLATFORM (  
->   ActorID VARCHAR(10),  
->   PlatformID VARCHAR(10),  
->   PRIMARY KEY (ActorID, PlatformID)  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> INSERT INTO MOVIE_ACTOR VALUES  
-> ('M101','A101'),  
-> ('M101','A102'),  
-> ('M102','A101'),  
-> ('M102','A103'),  
-> ('M103','A102');  
Query OK, 5 rows affected (0.01 sec)  
Records: 5 Duplicates: 0 Warnings: 0  
  
mysql> INSERT INTO MOVIE_PLATFORM VALUES  
-> ('M101','P01'),  
-> ('M102','P01'),  
-> ('M102','P02'),  
-> ('M103','P01');  
Query OK, 4 rows affected (0.01 sec)  
Records: 4 Duplicates: 0 Warnings: 0  
  
mysql> INSERT INTO ACTOR_PLATFORM VALUES  
-> ('A101','P01'),  
-> ('A102','P01'),  
-> ('A103','P02');  
Query OK, 3 rows affected (0.01 sec)  
Records: 3 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM MOVIE_ACTOR;
+-----+-----+
| MovieID | ActorID |
+-----+-----+
| M101    | A101    |
| M101    | A102    |
| M102    | A101    |
| M102    | A103    |
| M103    | A102    |
+-----+-----+
5 rows in set (0.00 sec)

mysql> SELECT * FROM MOVIE_PLATFORM;
+-----+-----+
| MovieID | PlatformID |
+-----+-----+
| M101    | P01        |
| M102    | P01        |
| M102    | P02        |
| M103    | P01        |
+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT * FROM ACTOR_PLATFORM;
+-----+-----+
| ActorID | PlatformID |
+-----+-----+
| A101    | P01        |
| A102    | P01        |
| A103    | P02        |
+-----+-----+
3 rows in set (0.00 sec)
```

LOSSLESS-JOIN VERIFICATION

```
mysql> SELECT MA.MovieID,
->      MA.ActorID,
->      MP.PlatformID
-> FROM MOVIE_ACTOR MA
-> JOIN MOVIE_PLATFORM MP
->   ON MA.MovieID = MP.MovieID
-> JOIN ACTOR_PLATFORM AP
->   ON MA.ActorID = AP.ActorID
->   AND MP.PlatformID = AP.PlatformID;
+-----+-----+-----+
| MovieID | ActorID | PlatformID |
+-----+-----+-----+
| M101    | A102    | P01        |
| M101    | A101    | P01        |
| M102    | A101    | P01        |
| M102    | A103    | P02        |
| M103    | A102    | P01        |
+-----+-----+-----+
5 rows in set (0.01 sec)
```

Thus,

MOVIE_STREAMING =

MOVIE_ACTOR

⋈ MOVIE_PLATFORM

⋈ ACTOR_PLATFORM

RESULT:

Thus, the Movie Streaming relation was successfully decomposed into 5NF by identifying the non-trivial join dependency and separating the movie–actor, movie–platform, and actor–platform relationships. The original relation can be reconstructed through natural joins, confirming that the decomposition is lossless.